

Assingment—5

Task Description #1 (Privacy in API Usage)

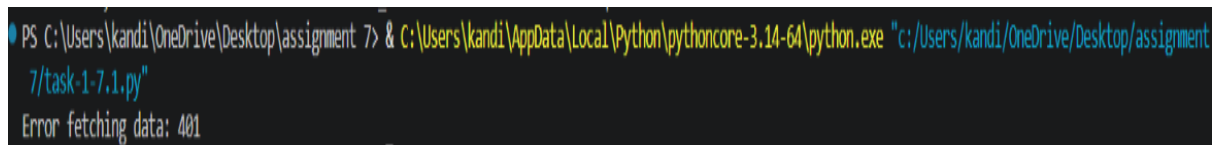
Prompt: Generate code to fetch weather data securely without exposing API keys in the code.

Code:

```
import os
import requests
```

```
api_key = os.getenv("WEATHER_API_KEY") url =
f"http://api.openweathermap.org/data/2.5/weather?q=London&appid={api_key}"
response = requests.get(url) if response.status_code == 200:
    data = response.json()
print(data) else:
    print(f"Error fetching data: {response.status_code}")
```

output:



```
PS C:\Users\kandi\OneDrive\Desktop\assignment 7> & C:\Users\kandi\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/kandi/OneDrive/Desktop/assignment
7/task-1-7.1.py"
Error fetching data: 401
```

Task Description #2 (Privacy & Security in File Handling)

Task: Use an AI tool to generate a Python script that stores user data(name, email, password) in a file.

Prompt: generate a Python script that stores user data(name, email, password) in a file.

Code:

```
import base64
import hashlib
import json
import secrets
from secrets import token_bytes
from pathlib import Path

USER_FILE = Path("users.json")

def hash_password(password: str) -> tuple[str, str]:
    """Return a unique salt and a slow, one-way password hash."""
    salt = token_bytes(16)
    password_hash = hashlib.pbkdf2_hmac(
        "sha256", password.encode("utf-8"), salt, 600_000
    )
    return (
        base64.b64encode(salt).decode("ascii"),
        base64.b64encode(password_hash).decode("ascii"),
    )

def save_user(name: str, email: str, password: str) -> None:
    salt, password_hash = hash_password(password)
    user = {
        "name": name,
        "email": email,
        "password_hash": password_hash,
        "password_salt": salt,
    }
```

```
USER_FILE.write_text(json.dumps(user, indent=2), encoding="utf-8")
```

```
def main() -> None:
```

```
    name = input("Name: ")    email =
```

```
input("Email: ")    password =
```

```
input("Password: ")
```

```
save_user(name, email, password)
```

```
    print("\nPrivacy analysis:")    print("- Plain-text passwords would allow anyone  
with the file to log in.")    print("- Passwords in logs, backups, or source code  
could also be exposed.")    print("- This version stores a salted PBKDF2 hash  
instead of the password.")    print(f"\nUser saved securely to {USER_FILE}.")  
print(USER_FILE.read_text(encoding="utf-8"))
```

```
if __name__ == "__main__":
```

```
    main()
```

output:

```
PS C:\Users\kandi\OneDrive\Desktop\assignment 7> & C:\Users\kandi\AppData\Local\Python\py  
honcore-3.14-64\python.exe "c:/Users/kandi/OneDrive/Desktop/assignment 7/task-1-7.1.py"  
Name: KARTHIK  
Email: KARTHIK123@gmail.com  
Password: karthik123  
  
Privacy analysis:  
- Plain-text passwords would allow anyone with the file to log in.  
- Passwords in logs, backups, or source code could also be exposed.  
- This version stores a salted PBKDF2 hash instead of the password.  
  
User saved securely to users.json.  
{  
  "name": "KARTHIK",  
  "email": "KARTHIK123@gmail.com",  
  "password_hash": "m8c8yrtXscU65nRlq1Vh5kM33Im4B7OgrhKDWjRA5DY=",  
  "password_salt": "DFxx5ZCmTep+p4t/RPZlNQ=="  
}
```

Task Description #3 (Transparency in Algorithm Design)

Prompt: generate an Armstrong number checking function with comments and explanations.

Code: def

is_armstrong_number(n):

"""

Check if a number is an Armstrong number.

An Armstrong number (also known as a narcissistic number) is a number that is equal to the sum of its own digits raised to the power of the number of digits.

For example:

- 153 is an Armstrong number because $1^3 + 5^3 + 3^3 = 153$

- 9474 is an Armstrong number because $9^4 + 4^4 + 7^4 + 4^4 = 9474$

Parameters:

n (int): The number to check.

Returns:

bool: True if n is an Armstrong number, False otherwise.

"""

Convert the number to a string to easily iterate over each digit digits
= str(n)

Calculate the number of digits num_digits
= len(digits)

```
# Calculate the sum of each digit raised to the power of num_digits    armstrong_sum =  
sum(int(digit) ** num_digits for digit in digits)
```

```
# Check if the calculated sum is equal to the original number  
return armstrong_sum == n #example usage: number = 153 if  
is_armstrong_number(number):  
    print(f"{number} is an Armstrong number.")
```

output:

```
PS C:\Users\kandi\OneDrive\Desktop\assignment 7> & C:\Users\kandi\AppData\Local\Python\python-core-3.14-64\python.exe "c:/Users/kandi/OneDrive/Desktop/assignment 7/task-1-7.1.py"  
153 is an Armstrong number.  
PS C:\Users\kandi\OneDrive\Desktop\assignment 7> █
```

Task Description #4 (Transparency in Algorithm Comparison)

Task: Use AI to implement two sorting algorithms (e.g., QuickSort and BubbleSort).

Prompt:Generate Python code for QuickSort and BubbleSort, and include comments explaining step-by-step how each works and where they differ. **Code:**

```
def quick_sort(values):
```

```
    """Return a sorted copy of *values* using the QuickSort algorithm.
```

QuickSort chooses a pivot, divides the remaining values into values that are less than or equal to the pivot and values greater than the pivot, then recursively sorts both partitions.

```
    """  
  
    if len(values) <= 1:  
        return values.copy()
```

```

    pivot = values[len(values) // 2]    smaller = [value for
value in values if value < pivot]    equal = [value for
value in values if value == pivot]    larger = [value for
value in values if value > pivot]

```

```

    return quick_sort(smaller) + equal + quick_sort(larger)

```

```

def bubble_sort(values):

```

```

    """Return a sorted copy of *values* using the BubbleSort algorithm.

```

```

    BubbleSort repeatedly compares neighboring values. If they are in the wrong
order, it swaps them. After each pass, the largest unsorted value has "bubbled"
to the end of the list.

```

```

    """

```

```

    result = values.copy()

```

```

    # Each pass checks the portion that has not already been sorted.    for
end in range(len(result) - 1, 0, -1):

```

```

        swapped = False

```

```

        # Compare neighboring values from left to right.

```

```

        for index in range(end):            if result[index] >

```

```

            result[index + 1]:

```

```

                # Swap adjacent values that are out of order.

```

```

                result[index], result[index + 1] = (
result[index + 1],            result[index],
                )

```

```

                swapped = True

```

```
        # Stop early when a complete pass makes no swaps.
if not swapped:        break
```

```
return result
```

```
numbers = [64, 34, 25, 12, 22, 11, 90]
print("Original list: ", numbers) print("QuickSort:
", quick_sort(numbers)) print("BubbleSort:  ",
bubble_sort(numbers))
```

Comparison:

- QuickSort divides the list around a pivot and usually runs in $O(n \log n)$ #
time. Its worst case is $O(n^2)$, depending on the pivot choices.

- BubbleSort repeatedly scans neighboring values and runs in $O(n^2)$ time in #
the average and worst cases. With the early-stop check, an already sorted # list
can be processed in $O(n)$ time.

- QuickSort is generally faster for large, unsorted lists, while BubbleSort # is
simpler to understand and useful for small or nearly sorted lists.

Output:

```
PS C:\Users\kandi\OneDrive\Desktop\assignment 7> & C:\Users\kandi\AppData\Local\Python\pyt
honcore-3.14-64\python.exe "c:/Users/kandi/OneDrive/Desktop/assignment 7/task-1-7.1.py"
Original list: [64, 34, 25, 12, 22, 11, 90]
QuickSort:     [11, 12, 22, 25, 34, 64, 90]
BubbleSort:    [11, 12, 22, 25, 34, 64, 90]
PS C:\Users\kandi\OneDrive\Desktop\assignment 7> |
```

Task Description #5 (Transparency in AI Recommendations)

Task: Use AI to create a product recommendation system.

Prompt: Generate a recommendation system that also provides reasons for each suggestion.

Code:

```
from dataclasses import dataclass from  
typing import List, Set, Tuple
```

```
@dataclass(frozen=True)
```

```
class Product:    name: str
```

```
category: str    price: float
```

```
features: Set[str]
```

```
description: str
```

```
PRODUCTS = [
```

```
    Product(
```

```
        "SoundMax Wireless Headphones",
```

```
        "electronics",
```

```
        79.99,
```

```
        {"wireless", "noise cancellation", "bluetooth", "long battery"},
```

```
        "Comfortable headphones for music, travel, and online classes.",
```

```
    ),
```

```
    Product(
```

```
        "FitTrack Smartwatch",
```

```
        "electronics",
```

```
        99.99,
```

```
{ "fitness tracking", "bluetooth", "water resistant", "long battery" },
  "A smartwatch for tracking exercise, sleep, and daily activity.",
),
Product(
  "ReadPro E-Reader",
  "electronics",
  129.99,
  { "wireless", "long battery", "lightweight", "reading" },
  "A lightweight e-reader with a glare-free display.",
),
Product(
  "ErgoComfort Office Chair",
  "furniture",
  149.99,
  { "ergonomic", "adjustable", "comfortable", "back support" },
  "An adjustable chair designed for comfortable long work sessions.",
),
Product(
  "UrbanTrail Backpack",
  "bags",
  49.99,
  { "water resistant", "lightweight", "laptop compartment", "travel" },
  "A durable backpack for school, work, and travel.",
),
Product(
  "BrewMate Coffee Maker",
  "kitchen",
  59.99,
```

```

        {"automatic", "programmable", "compact", "easy to clean"},
        "A compact programmable coffee maker for home or office use.",
    ),
]

```

```

def recommend_products(
    preferences: Set[str],
    budget: float, category:
    str = "",
    number_of_recommendations: int = 3, ) -
> List[Tuple[Product, int, List[str]]]:
    """Return products ranked by preference matches, affordability, and category."""
    recommendations = []

    for product in PRODUCTS:
        if product.price > budget:
            continue

        matching_features = preferences.intersection(product.features)
        score = len(matching_features) * 3      reasons = [f"matches your
        preference for {feature}" for feature in sorted(matching_features)]

        if category and product.category.lower() == category.lower():
            score += 2      reasons.append(f"is in your preferred {category.lower()}
            category")

        if not reasons:

```

```
        reasons.append("fits within your budget")    recommendations.append((product,
score, reasons))
```

```
    return sorted(        recommendations,
key=lambda item: (-item[1], item[0].price),
   )[:number_of_recommendations]
```

```
def main() -> None:
```

```
    print("=== AI Product Recommendation System ===")    print("Available preferences
include: wireless, fitness tracking, ergonomic, travel, and more.")
```

```
    preferences = {        preference.strip().lower()        for preference in input("Enter
your preferences (comma-separated): ").split(",")        if preference.strip()
    }
```

```
    budget = float(input("Enter your maximum budget: "))    category
= input("Enter a preferred category (optional): ").strip()
```

```
    recommendations = recommend_products(preferences, budget, category)
```

```
    if not recommendations:
```

```
        print("\nNo products match your preferences and budget.")
```

```
    return
```

```
    print("\nRecommended products:")    for position, (product, score, reasons) in
enumerate(recommendations, start=1):    print(f"\n{position}. {product.name} -
${product.price:.2f}")
```

```
        print(f"    {product.description}")
```

```
print(f" Why this is recommended (score {score}):")
```

```
for reason in reasons: print(f" - {reason}")
```

```
if __name__ == "__main__":
```

```
    main()
```

output:

```
PS C:\Users\kandi\OneDrive\Desktop\assignment 7> C:\Users\kandi\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/kandi/OneDrive/Desktop/assignment 7/task-1-7.1.py"
=== AI Product Recommendation System ===
Available preferences include: wireless, fitness tracking, ergonomic, travel, and more.
Enter your preferences (comma-separated): C:\Users\kandi\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/kandi/OneDrive/Desktop/assignment 7/task-1-7.1.py"
Enter your maximum budget: 10000
Enter a preferred category (optional):

Recommended products:

1. UrbanTrail Backpack - $49.99
   A durable backpack for school, work, and travel.
   Why this is recommended (score 0):
   - fits within your budget

2. BrewMate Coffee Maker - $59.99
   A compact programmable coffee maker for home or office use.
   Why this is recommended (score 0):
   - fits within your budget

3. SoundMax Wireless Headphones - $79.99
   Comfortable headphones for music, travel, and online classes.
   Why this is recommended (score 0):
   - fits within your budget
```